Solution: Problem Challenge 1: Palindrome LinkedList

Problem Statement

Given the head of a **Singly LinkedList**, write a method to check if the **LinkedList is a palindrome or not.**

Your algorithm should use **constant space** and the input LinkedList should be in the original form once the algorithm is finished. The algorithm should have O(N) time complexity where 'N' is the number of nodes in the LinkedList.

Example 1:

```
Input: 2 -> 4 -> 6 -> 4 -> 2 -> null
Output: true
```

Example 2:

```
Input: 2 -> 4 -> 6 -> 4 -> 2 -> 2 -> null
Output: false
```

Solution

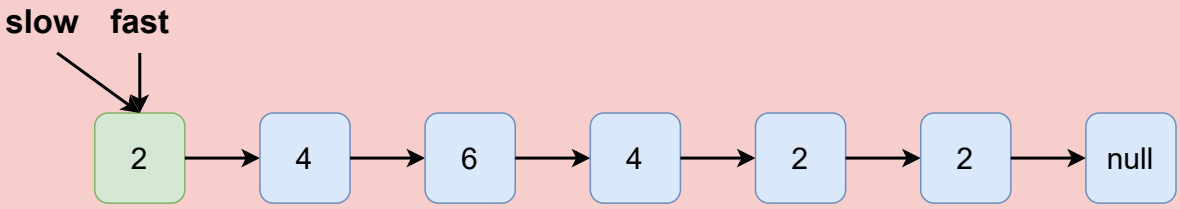
As we know, a palindrome LinkedList will have nodes values that read the same backward or forward. This means that if we divide the LinkedList into two halves, the node values of the first half in the forward direction should be similar to the node values of the second half in the backward direction. As we have been given a Singly LinkedList, we can't move in the backward direction. To handle this, we will perform the following steps:

1. We can use the **Fast** & **Slow** pointers method similar to Middle of the LinkedList to find the middle node of the LinkedList.
2. Once we have the middle of the LinkedList, we will reverse the second half.
3. Then, we will compare the first half with the reversed second half to see if the LinkedList represents a palindrome.
4. Finally, we will reverse the second half of the LinkedList again to revert and bring the LinkedList back to its original form.

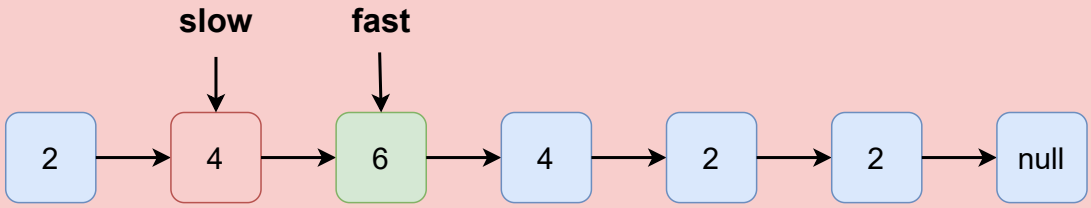
Here is the visual representation of this algorithm for Example-2:

To check whether linked list is palindrome,
1. find middle of the list
2. reverse the second half
3. compare with the first half.

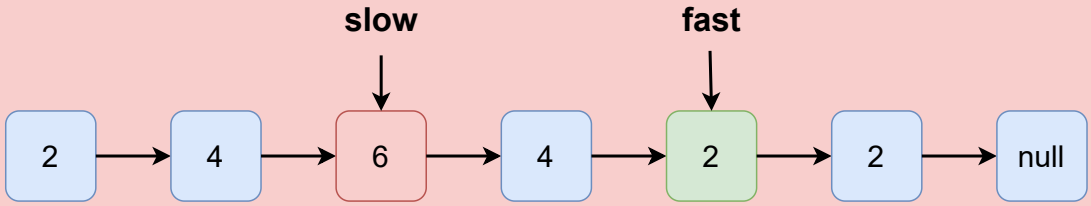
Finding middle of the list using slow and fast pointer.

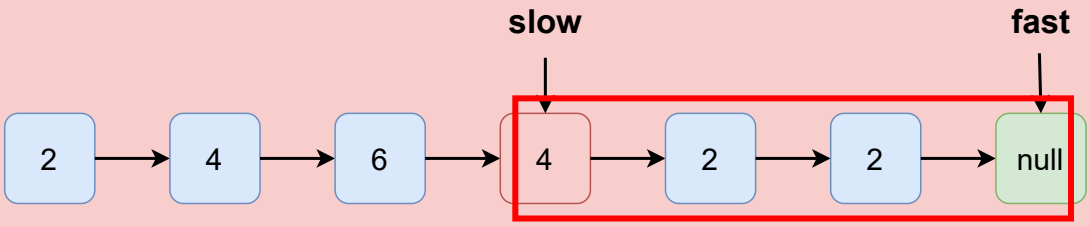


Move slow pointer one step, and fast pointer two step in each iteration.



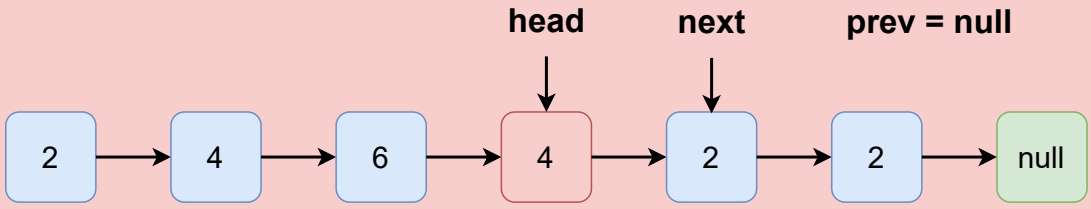
Again, move slow pointer one step, and fast pointer two step in each iteration.





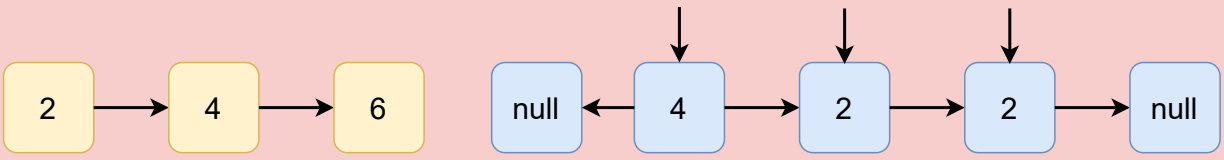
fast = null. So, 'slow' is middle of the linked list.

Reverse the second half of the list.

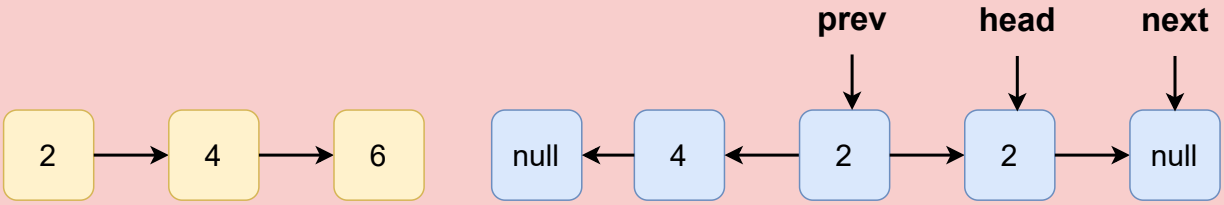


update head.next = prev = null, prev = head, head = next to reverse the list.

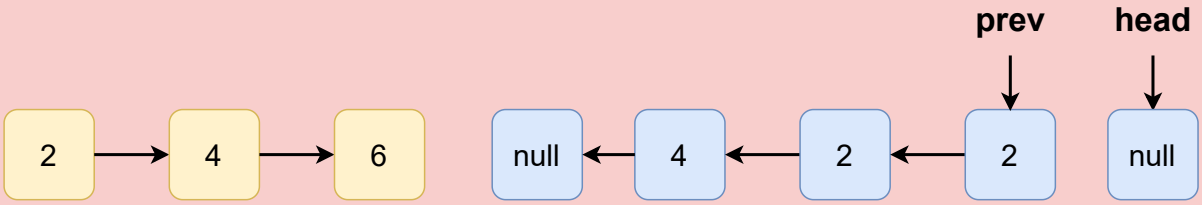
prev head next



Follow same procedure again as above step.

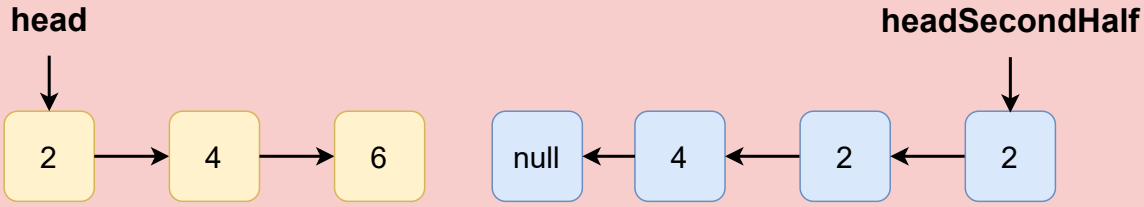


Follow same procedure again as above step.

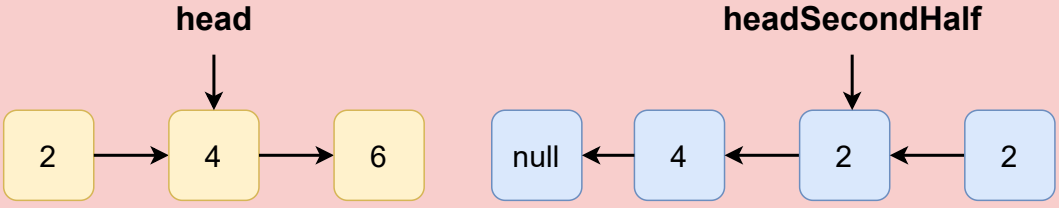


Second half is reversed.

Compare reversed second half with first half.



head.val == headSecondHalf.val. So, move ahead.



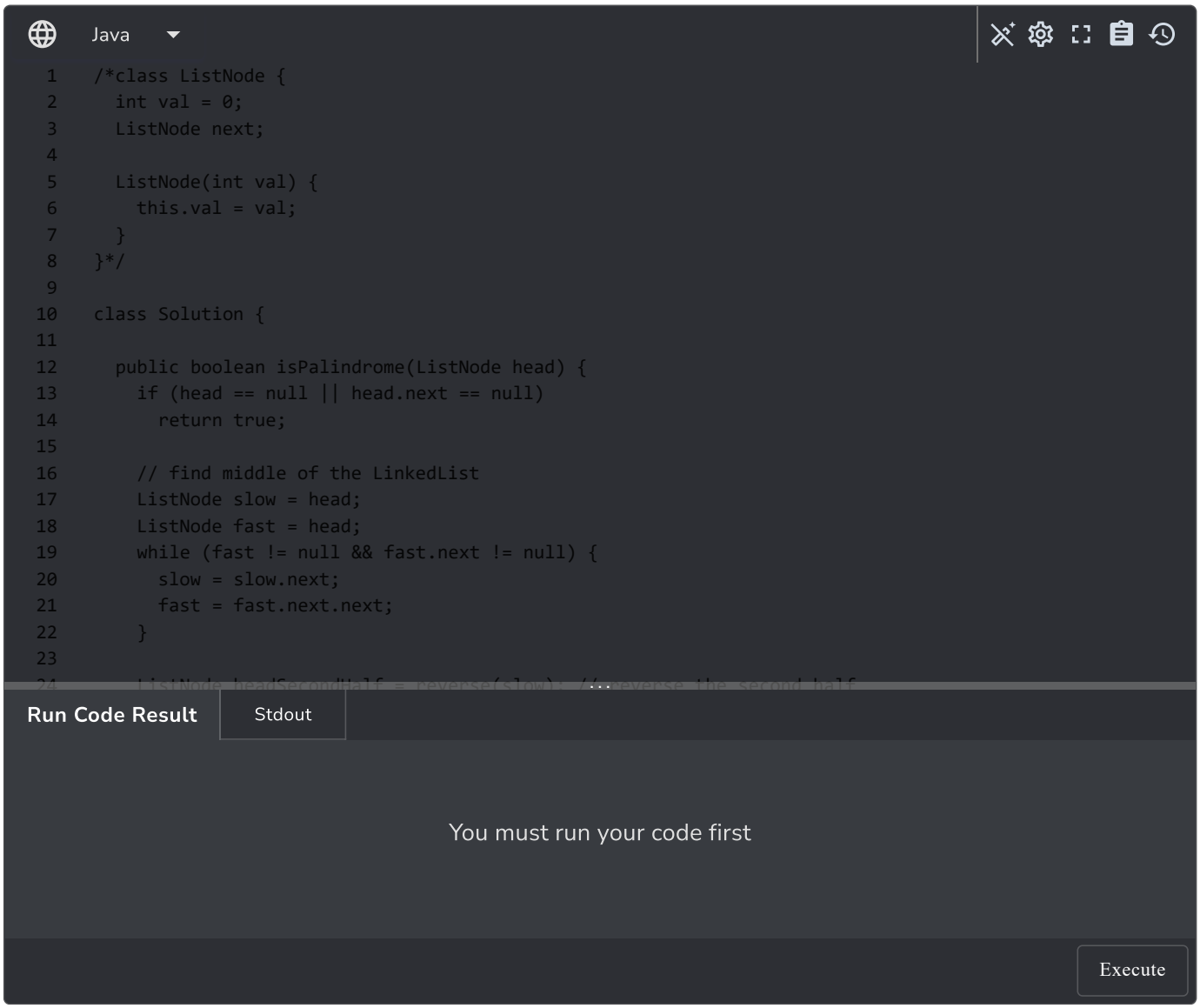
head.val != headSecondHalf.val. So, break the loop.

Reverse secondHalf agin to get original list.

head is not null, it means list is not palindrome.

Code

Here is what our algorithm will look like:



Time Complexity

The above algorithm will have a time complexity of O(N) where 'N' is the number of nodes in the LinkedList.

Space Complexity

The algorithm runs in constant space O(1).

← Previous

Next →